

FRED TALKS

8th October 2026

CONTENTS

Every layer of review makes you 10x slower

AVERY PENNARUN · 2841 WORDS

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 1776 WORDS

The End of Code Review? Or an Opportunity to Rethink it?

CHRISTIAN KÄSTNER · 2579 WORDS

2.1.294

CHANGELOG · 48 WORDS

Every layer of review makes you 10x slower

AVERY PENNARUN · 26 MAR 2026 · [SOURCE](#)

2026-03-16 »

Every layer of review makes you 10x slower

We've all heard of those network effect laws: the value of a network goes up with the square of the number of members. Or the cost of communication goes up with the square of the number of members, or maybe it was $n \log n$, or something like that, depending how you arrange the members. Anyway doubling a team doesn't double its speed; there's coordination overhead. Exactly how much overhead depends on how badly you botch the org design.

But there's one rule of thumb that someone showed me decades ago, that has stuck with me ever since, because of how annoyingly true it is. The rule is annoying because it doesn't *seem* like it should be true. There's no theoretical basis for this claim that I've ever heard. And yet, every time I look for it, there it is.

Here we go:

Every layer of approval makes a process 10x slower

I know what you're thinking. Come on, 10x? That's a lot. It's unfathomable. Surely we're exaggerating.

Nope.

Just to be clear, we're counting "wall clock time" here rather than effort. Almost all the extra time is spent sitting and waiting.

Look:

- Code a simple bug fix
30 minutes
- Get it code reviewed by the peer next to you
300 minutes → 5 hours → half a day

- Get a design doc approved by your architects team first
50 hours → about a week
- Get it on some other team's calendar to do all that
(for example, if a customer requests a feature)
500 hours → 12 weeks → one fiscal quarter

I wish I could tell you that the next step up — 10 quarters or about 2.5 years — was too crazy to contemplate, but no. That's the life of an executive sitting above a medium-sized team; I bump into it all the time even at a relatively small company like Tailscale if I want to change product direction. (And execs sitting above large teams can't actually do work of their own at all. That's another story.)

AI can't fix this

First of all, this isn't a post about AI, because AI's direct impact on this problem is minimal. Okay, so Claude can code it in 3 minutes instead of 30? That's super, Claude, great work.

Now you either get to spend 27 minutes reviewing the code yourself in a back-and-forth loop with the AI (this is actually kinda fun); or you save 27 minutes and submit unverified code to the code reviewer, who will still take 5 hours like before, but who will now be mad that you're making them read the slop that you were too lazy to read yourself. Little of value was gained.

Now now, you say, that's not the value of agentic coding. You don't use an agent on a 30-minute fix. You use it on a monstrosity week-long project that you and Claude can now do in a couple of hours! Now we're talking. Except no, because the monstrosity is so big that your reviewer will be extra mad that you didn't read it yourself, and it's too big to review in one chunk so you have to slice it into new bite-sized chunks, each with a 5-hour review cycle. And there's no design doc so there's no intentional architecture, so eventually someone's going to push back on that and here we go with the design doc review meeting, and now your monstrosity week-long project that you did in two hours is... oh. A week, again.

I guess I could have called this post Systems Design 4 (or 5, or whatever I'm up to now, who knows, I'm writing this on a plane with no wifi) because yeah, you guessed it. It's Systems Design time again.

The only way to sustainably go faster is fewer reviews

It's funny, everyone has been predicting the Singularity for decades now. The premise is we build systems that are so smart that they themselves can build the next system that is even smarter, that builds the next smarter one, and so on, and once we get that started, if they keep getting smarter faster enough, then the incremental time (t) to achieve a unit (u) of improvement goes to zero, so (u/t) goes to infinity and foom.

Anyway, I have never believed in this theory for the simple reason we outlined above: the majority of time needed to get anything done is not actually the time doing it. It's wall clock time. Waiting. Latency.

And you can't overcome latency with brute force.

I know you want to. I know many of you now work at companies where the business model kinda depends on doing exactly that.

Sorry.

But you can't just *not* review things!

Ah, well, no, actually yeah. You really can't.

There are now many people who have seen the symptom: the start of the pipeline (AI generated code) is so much faster, but all the subsequent stages (reviews) are too slow! And so they intuit the obvious solution: stop reviewing then!

The result might be slop, but if the slop is 100x cheaper, then it only needs to deliver 1% of the value per unit and it's still a fair trade. And if your value per unit is even a mere 2% of what it used to be, you've doubled your returns! Amazing.

There are some pretty dumb assumptions underlying that theory; you can imagine them for yourself. Suffice it to say that this produces what I will call the AI Developer's Descent Into Madness:

1. Whoa, I produced this prototype so fast! I have super powers!
2. This prototype is getting buggy. I'll tell the AI to fix the bugs.
3. Hmm, every change now causes as many new bugs as it fixes.
4. Aha! But if I have an AI agent also review the code, it can find its own bugs!
5. Wait, why am I personally passing data back and forth between agents

6. I need an agent framework

7. I can have my agent write an agent framework!

8. Return to step 1

It's actually alarming how many friends and respected peers I've lost to this cycle already. Claude Code only got good maybe a few months ago, so this only recently started happening, so I assume they will emerge from the spiral eventually. I mean, I hope they will. We have no way of knowing.

Why we review

Anyway we know our symptom: the pipeline gets jammed up because of too much new code spewed into it at step 1. But what's the root cause of the clog? Why doesn't the pipeline go faster?

I said above that this isn't an article about AI. Clearly I'm failing at that so far, but let's bring it back to humans. It goes back to the annoyingly true observation I started with: every layer of review is 10x slower. As a society, we know this. Maybe you haven't seen it before now. But trust me: people who do org design for a living know that layers are expensive... and they still do it.

As companies grow, they all end up with more and more layers of collaboration, review, and management. Why? Because otherwise mistakes get made, and mistakes are increasingly expensive at scale. The average value added by a new feature eventually becomes lower than the average value lost through the new bugs it causes. So, lacking a way to make features produce more value (wouldn't that be nice!), we try to at least reduce the damage.

The more checks and controls we put in place, the slower we go, but the more monotonically the quality increases. And isn't that the basis of *continuous improvement*?

Well, sort of. Monotonically increasing quality is on the right track. But "more checks and controls" went off the rails. That's *only one* way to improve quality, and it's a fraught one.

"Quality Assurance" reduces quality

I wrote a few years ago about W. E. Deming and the "new" philosophy around quality that he popularized in Japanese auto manufacturing. (Eventually U.S. auto manufacturers more or less got the idea. So far the software industry hasn't.)

One of the effects he highlighted was the problem of a “QA” pass in a factory: build widgets, have an inspection/QA phase, reject widgets that fail QA. Of course, your inspectors probably miss some of the failures, so when in doubt, add a second QA phase after the first to catch the remaining ones, and so on.

In a simplistic mathematical model this seems to make sense. (For example, if every QA pass catches 90% of defects, then after two QA passes you’ve reduced the number of defects by 100x. How awesome is that?)

But in the reality of agentic humans, it’s not so simple. First of all, the incentives get weird. The second QA team basically serves to evaluate how well the first QA team is doing; if the first QA team keeps missing defects, fire them. Now, that second QA team has little incentive to produce that outcome for their friends. So maybe they don’t look too hard; after all, the first QA team missed the defect, it’s not unreasonable that we might miss it too.

Furthermore, the first QA team knows there is a second QA team to catch any defects; if I don’t work too hard today, surely the second team will pick up the slack. That’s why they’re there!

Also, the team making the widgets in the first place doesn’t check their work too carefully; that’s what the QA team is for! Why would I slow down the production of every widget by being careful, at a cost of say 20% more time, when there are only 10 defects in 100 and I can just eliminate them at the next step for only a 10% waste overhead? It only makes sense. Plus they’ll fire me if I go 20% slower.

To say nothing of a whole engineering redesign to improve quality, that would be super expensive and we could be designing all new widgets instead.

Sound like any engineering departments you know?

Well, this isn’t the right time to rehash Deming, but suffice it to say, he was on to something. And his techniques worked. You get things like the famous Toyota Production System where they eliminated the QA phase entirely, but gave everybody an “oh crap, stop the line, I found a defect!” button.

Famously, US auto manufacturers tried to adopt the same system by installing the same “stop the line” buttons. Of course, nobody pushed those buttons. They were afraid of getting fired.

Trust

The basis of the Japanese system that worked, and the missing part of the American system that didn't, is trust. Trust among individuals that your boss Really Truly Actually wants to know about every defect, and wants you to stop the line when you find one. Trust among managers that executives were serious about quality. Trust among executives that individuals, given a system that can work and has the right incentives, will produce quality work and spot their own defects, and push the stop button when they need to push it.

But, one more thing: trust that the system *actually does work*. So first you need a system that will work.

Fallibility

AI coders are fallible; they write bad code, often. In this way, they are just like human programmers.

Deming's approach to manufacturing didn't have any magic bullets. Alas, you can't just follow his ten-step process and immediately get higher quality engineering. The secret is, you have to get your engineers to engineer higher quality into the whole system, from top to bottom, repeatedly. Continuously.

Every time something goes wrong, you have to ask, "How did this happen?" and then do a whole post-mortem and the Five Whys (or however many Whys are in fashion nowadays) and fix the underlying Root Causes so that it doesn't happen again. "The coder did it wrong" is never a root cause, only a symptom. Why was it possible for the coder to get it wrong?

The job of a code reviewer isn't to review code. It's to figure out how to obsolete their code review comment, that whole class of comment, in all future cases, until you don't need their reviews at all anymore.

(Think of the people who first created "go fmt" and how many stupid code review comments about whitespace are gone forever. Now that's engineering.)

By the time your review catches a mistake, the mistake has already been made. The root cause happened already. You're too late.

Modularity

I wish I could tell you I had all the answers. Actually I don't have much. If I did, I'd be first in line for the Singularity because it sounds kind of awesome.

I think we're going to be stuck with these systems pipeline problems for a long time. Review pipelines — layers of QA — don't work. Instead, they make you slower while hiding root causes. Hiding causes makes them harder to fix.

But, the call of AI coding is strong. That first, fast step in the pipeline is *so fast!* It really does feel like having super powers. I want more super powers. What are we going to do about it?

Maybe we finally have a compelling enough excuse to fix the 20 years of problems hidden by code review culture, and replace it with a real culture of quality.

I think the optimists have half of the right idea. Reducing review stages, even to an uncomfortable degree, is going to be needed. But you can't just reduce review stages without something to replace them. That way lies the Ford Pinto or any recent Boeing aircraft.

The complete package, the table flip, was what Deming brought to manufacturing. You can't half-adopt a "total quality" system. You need to eliminate the reviews *and* obsolete them, in one step.

How? You can fully adopt the new system, in small bites. What if some components of your system can be built the new way? Imagine an old-school U.S. auto manufacturer buying parts from Japanese suppliers; wow, these parts are so well made! Now I can start removing QA steps elsewhere because I can just assume the parts are going to work, and my job of "assemble a bigger widget from the parts" has a ton of its complexity removed.

I like this view. I've always liked small beautiful things, that's my own bias. But, you can assemble big beautiful things from small beautiful things.

It's a lot easier to build those individual beautiful things in small teams that trust each other, that know what quality looks like *to them*. They deliver their things to customer teams who can clearly explain what quality looks like *to them*. And on we go. Quality starts bottom-up, and spreads.

I think small startups are going to do really well in this new world, probably better than ever. Startups already have fewer layers of review just because they have fewer people. Some startups will figure out how to produce high quality components quickly; others won't and will fail. Quality by natural selection?

Bigger companies are gonna have a harder time, because their slow review systems are baked in, and deleting them would cause complete chaos.

But, it's not just about company size. I think engineering teams at any company can get smaller, and have better defined interfaces between them.

Maybe you could have multiple teams inside a company competing to deliver the same component. Each one is just a few people and a few coding bots. Try it 100 ways and see who comes up with the best one. Again, quality by evolution. Code is cheap but good ideas are not. But now you can try out new ideas faster than ever.

Maybe we'll see a new optimal point on the [monoliths-microservices continuum](#). Microservices got a bad name because they were too micro; in the original terminology, a “micro” service was exactly the right size for a “two pizza team” to build and operate on their own. With AI, maybe it's one pizza and some tokens.

What's fun is you can also use this new, faster coding to experiment with different module *boundaries* faster. *Features* are still hard for lots of reasons, but refactoring and automated integration testing are things the AIs excel at. Try splitting out a module you were afraid to split out before. Maybe it'll add some lines of code. But suddenly lines of code are cheap, compared to the coordination overhead of a bigger team maintaining both parts.

Every team has some monoliths that are a little too big, and too many layers of reviews. Maybe we won't get all the way to Singularity. But, we can engineer a much better world. Our problems are solvable.

It just takes trust.

Related

[Highlights on "quality," and Deming's work as it applies to software development](#) (2016)

[Systems design 3: LLMs and the semantic revolution](#) (2025)

Unrelated

[SimSWE part 2: The perils of multitasking](#) (2017)

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 08 JUL 2026 · [SOURCE](#)

You can also read this post on [Prime Radiant's blog](#).

Almost all of this post was written on June 1, before Anthropic shipped Claude Tag. Nothing I'm writing about uses (or is based on) Claude Tag

At [Prime Radiant](#), we've got a bunch of agents in our Slack. I want to tell you a bit about how we've started shipping some changes to internal production with an "agentic user in the loop." Letting an agent work directly with the implementer building its runloop and tooling is getting us higher-quality tools and experiences for the agent without a human in the middle of a game of telephone.

But first, a bit of backstory on how we're using non-coding agents at Prime Radiant.

One of them, Scribble, is just there to listen for when someone reports a problem or provides a new bit of information. It opens tickets for the issues and updates an internal wiki with the information. (It also logs our daily standups and keeps us honest about whether we finished what we said we were going to do the previous day.)

Another, Nora, is our junior go-to-market "person." Nora is built on top of Nanoclaw and is still learning her craft, but is surprisingly helpful, partially because she considers herself a team member, not just an assistant. I've put a bunch of time into tuning her prompting and constitution. She gets a ton of value out of knowing that she journals obsessively. (I should note that I don't tend to gender my agents, but Nora picked a name and a gender and was very clear about that and...who am I to argue?)

I think the most surprising interaction I've had with Nora was the morning I woke up to a slack DM telling me that I needed to be speaking at more conferences. (I do.) She pointed out that the CFP for a particular AI conference had closed a week earlier, but that the program chair was known to take late submissions of interesting talks by DM. She'd taken the liberty of drafting a DM for me to send. I didn't actually *send* the DM, but I should have.

We have another agent, "Spec-together" that was a first prototype of what has become our new multi-player [Brainstorm](#) app.

And we have a small fleet of "Sen" personal assistant agents. They're **claw-esque*, but date from before openclaw was a thing. Each one has been trained to be an executive assistant, using a set of skills built on some of the resources that excellent human EAs use to learn their craft. Sen triages my mail, puts together a daily brief, does research tasks (both for me and independently), interacts with Linear, etc.

They have the standard set of affordances - They can chat on Slack. They use tools. They've got a task scheduler that can wake them up and re-inject a prompt they set for themselves at a specific time or on a schedule they define. They can use and author skills. Mine has a dropbox folder it shares with me. When we moved Sen to the cloud, I lost one of my favorite dumb features - Sen used to be able to print things directly. Sen (v1) was built on top of the Claude Agents SDK.

For the past month or so, I've been doing initial bringup of a "new" iteration of Sen, intended to be more of a colleague than an assistant. This time around, I'm building it from the ground up on top of an agents SDK I control. Lace got its start in May 2025 as my take on a command line coding agent. Over the past year, it's mutated a few times, first growing a web interface, then flipping to an ACP-derived wire protocol so you can connect any client you want to it. It speaks a bunch of different flavors of model provider API, manages caching, subagents, tools, skills, and all the other stuff you might expect. It has supported subagents in isolated containers for the better part of a year. One of the latest things to land has been the ability to run subagents locally and to project *all* of their tools into a container, such that the agent believes it is running in that container.

The thing I've been working on over the past few weeks is the credentials story for the new Sen agent running on top of Lace. Right now, I'm building with a design that assumes the agent has its own credentials for any service that it accesses, not yours. (You wouldn't share your email account or GitHub credentials with a colleague, right?) As of now, nobody has solved Simon's Lethal Trifecta - If a single agent has access to private information, can communicate externally, and is exposed to untrusted content, there is no structural way to guarantee that that agent can't be suborned. So the name of the game is compartmentalization and risk *reduction*.

The architecture I've landed on for now:

- The main agent does not have the ability to directly communicate externally.
- The main agent is able to communicate with ephemeral subagents that *do* have the ability to communicate externally

- No agent has intentional direct exposure to credentials (with one very real gap that I haven't solved yet)

All credentials for the agent live in a 1Password vault. All outgoing https traffic goes through a onecli-style transparent MITM (man in the middle) proxy. When a subagent wants to do something that requires a credential, it runs a command that asks an *arbiter* agent running in another container for permission to use that credential. If the arbiter decides the request is reasonable, it provides the subagent with a random string they can use in place of the requested credential. When the agent uses that random string in an outbound request to the right remote host, the MITM proxy rewrites the random string into the real credential on the fly.

The case that's a little more complicated is when the agent wants to do something that requires a browser, like logging into Gmail as themselves or using a service that has no API. For a bunch of reasons, the MITM rewrite trick does not work as reliably in a browser context. Often the site/app's javascript plays games with the password to hash the password before sending it. Or does some other weird validation. Or is actually sending it to a different backend service. So far, the best I've come up with keeps the credential out of the agent's transcript and prevents unintentional disclosure. In these cases, the subagent running in the container generates a random temporary password, uses [superpowers-chrome](#) to fill it into the password field, and then runs a command that asks the arbiter agent (running in its own container) for help. The arbiter, if it decides the request is reasonable, instructs a helper tool to log into the subagent's container and replace the random password with the real one. It's not perfect and there's more we can do to lock things down, but it's a better first pass than anything I've had to date.

What actually got me writing today was not this architecture, but the *development pattern* I've been using to build this. On the one side, I've got a Claude Code session. I hooked it up to our Slack instance with a self-provisioned commandline slack client Drew built called [Slackline](#) - Slackline is designed to make it easy for agents that don't *live* on Slack to participate in discussions there. It has primitives for sending messages, reading chats, waiting for replies, etc. The whole command-line slackline tool has been built for agents as its primary user. Sure, a human *could* use it to use slack from the commandline. And sure, it has enough of a json api that you could use it as part of traditional automation. But it's *for* agents.

It took a bit of iteration to get to the point where the new Sen 2.0 agentic colleague harness was live on Slack. But once it was there, I could talk through its experience of its tools and memory and credentials and workspaces. And the issues showed up pretty quickly.

The "right" kind of context compaction for a coding agent is desperately wrong for a long-lived persona that might be engaging in multiple simultaneous conversations. And the only way to see if the credentials management tools work is to try to do something crazy, like sign into GitHub from a subagent session.

And sure, I could take this feedback into a Claude Code (or Codex) session and work it through. But I've got too many projects in flight and there's very little reason for me to be in the loop on most of this work. I would just slow it down. So I told Claude Code to use slackline to talk to "@Ada-sen" about the work it was doing.

Claude pinged Ada in #bot-testing to describe the work we'd just done and to ask Ada if it could test it out. Ada dutifully fired up a subagent and tried to log into GitHub. Claude watched the logs to see if the credential proxy did the right thing. It did not.

That kicked off a number of days of iteration. Claude would propose a change to Ada. Ada would review Claude's spec, raising questions or concerns. Claude would update the spec. Once Claude and Ada were in agreement, Claude would build an updated version of Ada, make sure Ada wasn't in the middle of something, and then deploy the update. Once Ada was up and running again, Claude would @Ada-sen, explain the test it wanted to run, and wait for results.

Once a bit of functionality works, Claude checks in with Ada to see what could make it simpler and easier to use. Ada typically kicks off some subagents to see whether they stumble through the new feature and then reports back.

At one point, I wrote to Claude "I've got to get to bed. When this project is done, check with Ada to see whatever quality of life features you could build." I woke up to a laundry list of about a dozen harness improvements. Overnight, Claude had solicited a wishlist from Ada. Ada ranked a bunch of requests. Claude ordered them and wrote out a spec for everything it was comfortable building. Ada reviewed the specs, then Claude built the features. Ada tested them out and requested changes. Claude updated the features until Ada was happy. Once Ada signed off, Claude merged the changes to main and wrote up an after-action report for me.

It's been pretty magical to watch.

The End of Code Review? Or an Opportunity to Rethink it?

CHRISTIAN KÄSTNER · 26 SEP 2026 · [SOURCE](#)

Code review as we understand it today will probably not survive the shift to AI-generated code, but is that a problem? Code review is under pressure from agents producing code at an incredible pace, and reactions to that are quite split. There are many who want to protect code review as a core human software engineering practice, who put in place strict policies to preserve it. Others argue that code review is effectively dead, because it cannot keep up with the pace of production and will not survive competitive pressures if it becomes a bottleneck. Many experiment with automating code review at least partially. Implicit in this discussion is often the assumption that our prior code review practices were a gold standard, something to save or something to mourn. But is that really the case?

$$C = P + L \times (1 - E) + R$$

C Total cost (in \$/LOC)

P Production cost (in \$/LOC)
↳ Agents produce code faster

L Liability (in \$/LOC): Rework + Harm
↳ Lower quality, more harm and rework
↳ Higher quality, less harm and rework
↳ Agents lowering rework costs
↳ Cheaper substitutes: tests, proofs

E Effectiveness of Reviews (in %)
↳ Productivity pressure
↳ Deceptive plausibility
↳ Cognitive debt

R Review cost (in \$/LOC)
↳ Automated code review

A sweet spot for code review, historically?

Code review did not always look the way it does now (or at least the way it looked until coding agents took off). The contemporary practice that we now all know – tool-mediated, CI-integrated, lightweight review of individual commits, usually integrated with testing, static analysis, and other automation – is what reviewers call **modern code review** to distinguish it from other forms of code review and inspection.

Modern code review is called *modern* in contrast to older practices that many junior engineers have probably never seen and that we barely teach these days, most popular among them *Fagan-style inspections*. These were heavyweight processes, popularized in the 70s and widely studied through the 80s and early 90s: Multiple reviewers read small fragments of code line by line before release (not incrementally while code is created), often meeting to talk it through and sometimes reading it aloud. Fagan-style inspection was widely taught and written about (e.g., Wiegiers’ book “[Peer Reviews in Software](#)”). [Studies](#) showed it to be the most effective

technique to find bugs we have, more effective than testing. But it is slow and expensive, easily costing more than 8 developer hours across 3 reviewers, a moderator, and the author for reviewing merely 400 lines of code. That cost limited how much code could be reviewed. Fagan-inspections were well known in principle but only sparingly used for high-stakes settings – often to review the critical core of a product.

The lightweight practice that eventually took over grew out of patch-by-patch review in open-source projects and converged, remarkably, on nearly the same practice in companies and in GitHub’s pull requests. Studies are clear that modern code review is pretty bad at catching bugs, catching mostly shallow ones and readability issues (though better tooling has given reviewers a chance to go deeper). However, compared to Fagan-style inspections, it is cheap enough to do for *all* changes. While modern code review is not great at catching bugs, it provides many other benefits: forcing “developers to write code that other developers could understand,” fostering collective code ownership and awareness, supporting learning and mentorship, surfacing optimization opportunities, and enforcing conventions. Review has shifted, in Rigby’s phrase, from a defect-finding activity to a group problem-solving one.

There may be many plausible reasons for the almost universal adoption of modern code reviews – whether companies copying successful open source practices (coined “inner source”) or just wanting to be more like Google, whether inertia, or whether developers just do not trust their interns – but let’s assume somewhat rational actors: If modern code review wouldn’t have provided any benefits, we would have stopped spending so much effort on it. For the sake of this argument, let’s assume, the community has eventually settled on an **equilibrium** where the value modern code review provides just justified its cost in a pre-AI world, whether through catching bugs or through those other benefits.

A simple model to untangle competing arguments

Before looking at how agentic coding puts code review under pressure, let’s posit a simple model that will help to untangle various moving parts to help put various forces into perspective.

- **P**: The cost of producing code, in \$ per line.
- **L**: The liability of that code, in \$ per line, meaning the expected future cost of its quality problems. Let’s assume liability has two parts: (a) **Rework**, the cost of finding and fixing bugs or in the worst case rearchitect the entire system. (b) **Harm**, the cost when problems escape to the outside world as security incidents, reputational damage, and losses to

customers we may need to reimburse, sometimes after lawsuits. Rework is internal and is the main concern for low-stakes software, but harm is visible externally and can be substantial in high-stakes software.

- **R**: The cost of reviewing code, in \$ per line.
- **E**: The effectiveness of review at reducing liability, as a percentage.

Overall cost per line is then $P + L \times (1 - E) + R$. In a rational-actor world, teams invest in code review exactly up to the point where the last dollar spent on it just offsets the liability it removes.

For high-stakes code with high potential liability (L), it is rational to invest heavily (R) into making review more effective (E), up to Fagan-style inspections. For most code, stakes are not that high and liability comes mostly from rework. Also many other efforts have helped reduce liability without code review: Modern languages prevent whole classes of memory defects, static analysis catches many issues, and test generation and continuous-integration automation move testing into the cost of production. In that setting, shallow and fairly ineffective modern code review looks rational with modest cost for modest liability reduction.

Agentic coding shifts every part of the model, all at once

Agentic coding clearly has the potential to lower the cost of code production, but it can also influence the cost and effectiveness of code review, and the liability of typical code produced. The specific effects are often disputed, point in different directions, and interact with various things changing at the same time. The model can help us to untangle what we might expect to happen.

Production cost (P) is the obvious starting point but only part of the story. [↘ production cost] Agentic coding promises to drastically cut the cost of producing code (P). If nothing else changed (e.g., same code review, agentic code producing the same liability as human code), review cost (R) and liability cost (L) limit how much we can reduce overall cost. For example, even if production costs are reduced by 90%, overall costs would just be reduced by 30% if production accounted for only a third of the costs previously. This effect likely explains some of the discrepancies between frequent claims of 10X productivity improvement (which may even feel intuitively right seeing how quickly code is produced) and much lower productivity improvements measured in academic studies (rarely above 30%). If production cost is the only thing that changes, code review feels very much like a bottleneck limiting progress.

Agentic coding affects liability (L) in at least three plausible ways, and it is not obvious where we will land. When it comes to code quality and liability, the effect of coding agents is disputed.

[↗ liability due to lower code quality] First, there is some evidence (including our own study) that coding agents produce code of lower quality than humans, so that it needs more rework later. Also there are plenty of stories of harms from poor agentic code, like the multiple security incidents in the vibe-coded Moltbook and multiple AWS outages reported to be caused by agentic coding tools. All of that would point toward increased liability of code generated by coding agents, which could call for more scrutiny with code review.

[↘ liability due to better code quality] However, it is also plausible (even today with enough iteration and tool use, but plausibly more so with future improvements) that agentic code is of higher quality than human written code, lowering liability and the benefit of code review (E*L).

[↘ liability due to cheaper rework] Second, just like code production, agents promise to make *rework* cheaper too, with automation in debugging and repair, which shrinks the rework part of liability. Especially in low-stakes settings, rework is likely the larger part of the liability cost, and lower code quality may simply not matter much if it can be fixed easily later.

[↘ liability due to cheaper review substitutes] Third, alternatives to code review get cheaper too: Tests, static analysis, and automated code review, and maybe one day even formal verification and proof-carrying code at scale, can catch problems early during code production, reducing liability before code review. If such activities become effective and cheap enough, they render code review less important for liability reduction.

I do not think it is clear yet how this will pan out. In low-stakes settings, self-reviewing agents, more testing, and cheap rework may seriously reduce most liability, but it seems less obvious how it will affect the kind of downstream harms that matter in high-stakes settings – security issues, safety issues, financial losses. When issues can create real harms, patching a system in production after harms have occurred may not be the best strategy.

Productivity pressure, surface plausibility of agentic code, and cognitive debt can each undermine review effectiveness (E). [↘ effectiveness due to productivity pressure] Many practitioners report enormous pressure to improve productivity with AI. With more code arriving from agents and review becoming the bottleneck, this translates into pressure to review faster, which can result in rubber-stamping, undermining any effectiveness modern code review may still have had.

[↘ effectiveness due to deceptive plausibility of AI generated code and cognitive debt] In addition, agentic code may be harder to review well, because it looks plausible and well written (automation bias), and because a reviewer who was not engaged in producing it often lacks the context to judge it (cognitive debt). Both issues lower review effectiveness, or would require more investment into code review to retain effectiveness.

These effects make preserving traditional human code review particularly challenging if most code is generated by AI. Insisting on manual review, but pushing more reviews through faster, likely just hollows it out. I fully agree that this will not be sustainable and code review will not survive in this form.

Automating the code review may lower review cost (R). [↘ review cost with review automation] Many of the quality problems that surface in modern code review, especially the shallow ones modern code review catches, can be found by machines mimicking code review. *Automated code review* by agents can arguably find many of those kinds of issues and lower review costs similar to how AI lowered production costs. Some offerings of automated code review can be quite pricey (e.g., [Claude's code review costs \\$15-25 per reviewed PR](#)), but they still likely undercut the costs of humans and they don't share the human capacity bottleneck.

Lowered review costs from automated code review cannot be viewed in isolation. Whether automated code review is more or less effective (E) than humans, or effective only for particular classes of bugs, is still unsettled. Automation is likely able to go much deeper and be more systematic than a typical human review (closer to Fagan-style inspections with checklists), but it may also miss issues obvious to humans. If automation produces more false positives than human reviewers, it would push production cost (P) back up, though if production is cheap that may not matter much.

Indirect effects of code review, beyond finding bugs, may or may not matter, and they will decide the future of code review. Code review has many well understood benefits beyond its ability to find bugs, including learning and mentoring, (collective) code ownership, and general awareness – not captured in the simplified model (P, L, E, R). Pushing more code through the review process or automating code review away will obviously degrade those benefits too: If developers review less, they may gain less experience that helps them in producing code or being effective at those reviews still remaining. Developers may not understand their code base at all, making it harder for them to write good prompts to change it further, possibly increasing liability indirectly through cognitive debt. Or tooling can cover enough of what is lost, freeing up developers' time to focus on other parts that matter (e.g., requirements, specifications for test generation, infrastructure for testing in production).

Such effects are all plausible, but it is not clear to me how it will shake out. If we want to argue for code review, we should be clear about what benefits we want to gain from it (beyond finding bugs). And frankly, we should look for ways of getting those benefits without traditional code review. More research is needed, and in it we must be explicit about the mechanisms we expect and what effects we predict, whether on production cost, review cost, liability, or anything else.

Code review changed before

Heavyweight inspection never became the norm, because it cost too much to run everywhere, and the lighter practice that spread in its place exchanged rigor for scalability. I suspect code review will change again, or will be replaced by something looking completely different. The previous “modern code review” practices were probably close to an equilibrium, but agentic coding changes the forces in play, making that form of code review no longer sustainable.

Personally, I cannot predict where things will go. I think market pressures to increase productivity in an era of AI hype will make it difficult to preserve anything resembling our current modern code review practices when it becomes the bottleneck. But I also don’t see modern code review as a practice to protect for its own sake, just because that’s what we have previously done. I expect that coding agents will get much better at reviewing their own code during production, so that few of those more shallow and predictable issues remain that traditional code review or static analysis tools might have surfaced.

I think the bigger question is about all the other benefits we got from code review that were not about code quality, like coordination and mentoring. I think the better question to ask is how much those are still needed and how we can get them in forms that are not code review. For example, would better standup meetings or AI summaries of changes be able to replace the awareness benefits of code review? Would deliberate feedback and mentoring mechanisms better support the junior to senior software engineering pipeline than code review? I think stepping back and rethinking a world without code review is a more useful exercise than tweaking existing code review practices.

Zooming out further, I think eventually the big question is really about where and how and how much humans will be in control in some form when producing software. Maybe that control is better focused on requirements, architectural design, test design, and security/safety safeguards, rather than reviewing lines of AI-generated code.

So my main point is to stop asking whether and how code review in its current form can be saved, but to have an open conversation about what we are trying to achieve here and the costs and benefits of alternative models.

Acknowledgment: This model grew out of a conversation with Courtney Miller about whether preserving code review is the right goal for researchers to support. I am no longer sure it is.

Code review is probably on its last legs, and we should focus on how to retain the indirect benefits it provided...

2.1.294

CHANGELOG · 08 OCT 2026 · [SOURCE](#)

- Fixed `prompt` and `agent` hooks written as instructions (such as "Block commands that...") allowing what they should block
- Improved how `prompt` hooks on `Stop` and `SubagentStop` written as instructions (such as "Carry on if the build is broken") are judged, so Claude is less likely to stop early